OffBytes Lab

Prepared for:

Looputo Finance

October 21, 2026

Looputo Finance Audit Report

Table of Contents

1. Executive Summary 3
- About Looputo 3
- Risk Profile 3
- Audit Scope 3

2. Assumptions and Considerations 4
- Audit Assumptions 4
- Centralization Risks 4

3. Severity Definitions 5
- Impact 5
- Likelihood 5
- Severity Classification Matrix 5

4. Findings 7
- L01: Keeper batches can mutate arbitrary position lifecycle state 7
- L02: Partial close burns collateral claims and leaves residual assets claimable 8

5. About Us 10
- About OffBytes Lab 10
- Audit Methodology 10
- Confidentiality Notice 11
- Legal Disclaimer 11

© 2026 OffBytes Lab. All rights reserved.

1. Executive Summary

About Looputo

This audit focuses on Looputo Finance's leveraged-position pools, which let users deposit assets into a shared pool and take on leverage of their choosing while keepers monitor the pool to limit liquidation and negative-carry risk. Each pool tracks per-user collateral and debt shares and sources its collateral supply and debt borrowing from an Aave V3, Spark or Morpho Blue lending adapter, with position changes executed atomically through Morpho flash loans and swaps on owner-approved DEX aggregators. Pools are deployed as beacon proxies from an owner-controlled factory that fixes the assets, lending market, risk limits, fees, aggregators and keeper at initialization.

Risk Profile

The following table summarizes the distribution of identified vulnerabilities by risk level:

Risk Level	Count	Fixed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	0	0	0
Low	2	2	0

Audit Scope

- Repository: https://github.com/looputo-finance/Looputo-Finance
- Audit start date: September 30, 2026
- Commit hash: 3a37647d31192dc5f65ca6e2ff6fa679b5a7e8e4
- Files/Modules in Scope:
  - `src/LoopPool.sol`
  - `src/MorphoLoopPool.sol`
  - `src/AaveV3LoopPool.sol`
  - `src/SparkLoopPool.sol`
  - `src/Factory.sol`
  - `src/LoopPoolBeacon.sol`
  - `src/libraries/SharesLib.sol`
  - `src/libraries/MathLib.sol`

© 2026 OffBytes Lab. All rights reserved.

2. Assumptions and Considerations

Audit Assumptions

- I. Morpho Blue, Aave V3 and Spark core contracts are treated as trusted dependencies and behave as documented.
- II. DEX aggregators approved by the owner return the swapped assets to the pool and are not themselves malicious; swap calldata is supplied by the user and validated only by the pool's slippage checks.
- III. The collateral and debt tokens configured for a pool are standard ERC-20 tokens without fee-on-transfer or rebasing behaviour.

Centralization Risks

- IV. The `Factory` owner is a trusted, protocol-controlled address (governance multisig). It can change LTV limits, caps, fees, approved aggregators, the keeper, and upgrade every pool through the beacon.
- V. The keeper is a trusted, protocol-controlled operator that may batch `deleverage` / `releverage` user positions within the configured LTV bounds.
- VI. The fee collector is a passive protocol-controlled address.

© 2026 OffBytes Lab. All rights reserved.

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: Impact and Likelihood. These dimensions help project our team's understanding of both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

Impact

Impact reflects the potential consequences of the issue—particularly on project funds, user funds, and the availability or integrity of the protocol.

- **High Impact:** Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact:** Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact:** The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

- **High Likelihood:** The vulnerability is trivially exploitable. It can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risk.
- **Medium Likelihood:** The vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but with manageable financial risk.
- **Low Likelihood:** Exploitation is often technically unfeasible or requires highly specialized conditions. It may require extraordinary effort or significant financial risk for an attacker, with a high chance of failure and minimal potential return.

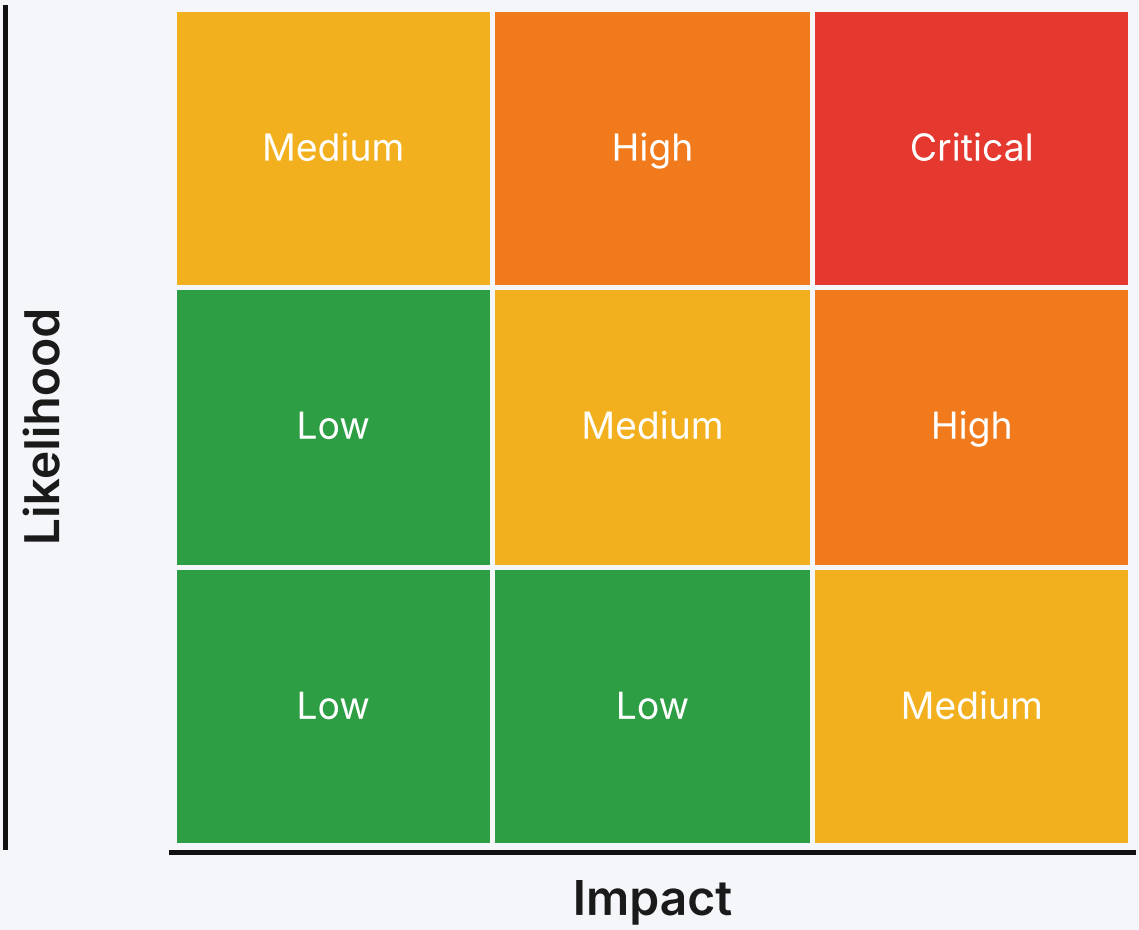
Severity Classification Matrix

By combining Impact and Likelihood, we assign a severity level using the matrix below:

- **Critical:** High impact + high likelihood (e.g. a bug that could allow anyone to drain a substantial amount of protocol funds with minimal effort)
- **High:** High impact with medium likelihood, or medium impact with high likelihood
- **Medium:** Moderate impact and/or discoverability
- **Low:** Minimal impact or unlikely to be exploited

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.

© 2026 OffBytes Lab. All rights reserved.



Severity Matrix

© 2026 OffBytes Lab. All rights reserved.

4. Findings

L01: Keeper batches can mutate arbitrary position lifecycle state

Status: Resolved

Impact: Medium

Likelihood: Low

Severity: Low

Location: https://github.com/looputo-finance/Looputo-Finance/blob/3a37647/src/LoopPool.sol#L1019-L1115

>_ LoopPool.sol SOLIDITY

```
1095:             address user = params.positions[i];
1096:             Position storage position = userPositions[user];
1097:
1098:             uint256 newCollShares = position.collShares + releveragedCollShares;
1099:             uint256 newDebtShares = position.debtShares + releveragedDebtShares;
1100:
1101:             position.collShares = newCollShares;
1102:             position.debtShares = newDebtShares;
1103:
1104:             position.status = PositionStatus.Active;
1105:
1106:             _checkUserPositionLTV(user);
```

Description:

The keeper-only `deleverage` and `releverage` entrypoints distribute the batch result through `_distributeDeleveragedShares` and `_distributeReleverageShares`, which iterate over the caller-supplied `positions` array and write directly into each corresponding `userPositions` record.

Neither helper validates the recipient. In `_distributeDeleveragedShares`, a non-final entry whose weight resolves to zero collateral and debt shares is still marked `Deleveraged` or `Unwound`; in `_distributeReleverageShares`, every listed address is marked `Active` after its allocation, including an allocation of zero. There is no check that the address holds a position in an eligible predecessor status, that it actually participated in the rebalance, or that it receives a nonzero share allocation. The per-user LTV check operates on the resulting share balances only, so a zero-share record passes it trivially.

This bypasses the normal `openPosition` flow, which requires an `Inactive` caller, a deposit and initialisation of position metadata before a record becomes `Active`.

Recommendation:

Before any status or share write in the two distributors, require that the listed address holds a position in an eligible predecessor status (`Active` / `Deleveraged` for deleverage; `Deleveraged` / `Unwound` with nonzero `collShares` for releverage) and that its weight produces a nonzero allocation. Reject zero-share and inactive recipients.

© 2026 OffBytes Lab. All rights reserved.

```
DIFF
            address user = params.positions[i];
            Position storage position = userPositions[user];
+           require(
+               position.status == PositionStatus.Active ||
+                   position.status == PositionStatus.Deleveraged,
+               "Position cannot be deleveraged"
+           );
+           require(
+               params.collAmounts[i] > 0 &&
+                   (deleveragedCollShares > 0 || deleveragedDebtShares > 0),
+               "Position must receive a deleverage allocation"
+           );
            uint256 newCollShares = position.collShares - deleveragedCollShares;
            ...
            address user = params.positions[i];
            Position storage position = userPositions[user];
+           require(
+               (position.status == PositionStatus.Deleveraged ||
+                   position.status == PositionStatus.Unwound) &&
+                   position.collShares > 0,
+               "Position cannot be releveraged"
+           );
+           require(
+               params.debtAmounts[i] > 0 &&
+                   (releveragedCollShares > 0 || releveragedDebtShares > 0),
+               "Position must receive a releverage allocation"
+           );
            uint256 newCollShares = position.collShares + releveragedCollShares;
```

L02: Partial close burns collateral claims and leaves residual assets claimable

Status: Resolved

Impact: Medium

Likelihood: Low

Severity: Low

Location: https://github.com/looputo-finance/Looputo-Finance/blob/3a37647/src/LoopPool.sol#L624-L688

© 2026 OffBytes Lab. All rights reserved.

>_ LoopPool.sol SOLIDITY

```
646:         require(
647:             params.swapAmount <= userCollAmount,
648:             "Required COLL_TOKEN exceeds available amount"
649:         );
650:
651:         totalCollShares -= userPositions[msg.sender].collShares;
652:         totalDebtShares -= userPositions[msg.sender].debtShares;
653:         delete userPositions[msg.sender];
654:
655:         uint256 debtTokenBalanceBefore = DEBT_TOKEN.balanceOf(address(this));
656:
657:         _executeFlashloan(
658:             false,
659:             params.swapAmount,
660:             0,
661:             params.aggregatorRouter,
662:             params.data,
663:             userDebtAmount
664:         );
```

Description:

`closePosition` accepts any `params.swapAmount` no greater than the caller's attributed collateral, but immediately removes all of the position's collateral and debt shares and deletes the record. It then flash-loans the full attributed debt, while the reduce branch of `onMorphoFlashLoan` withdraws and swaps only `amount + extraAmount` — and the close path passes `extraAmount` as `0`.

Consequently, a close that uses less than `userCollAmount` but still produces enough debt token to repay the flash loan succeeds, repays the full debt, and leaves the collateral difference in the lending market after the shares that owned it have been burned. The adapter's collateral cache continues to include that balance, so it is silently redistributed to the remaining share holders. If the close empties `totalCollShares`, the next `openPosition` mints shares 1:1 from its own collateral delta, while subsequent redemption values those shares against the full retained market balance — allowing the residual to be captured by the next opener.

Recommendation:

On close, withdraw any attributed collateral not consumed by the swap and return it to the closing user before deleting the position's claim, so a partial close can never leave unowned collateral in the lending market.

DIFF

```
        uint256 debtTokenBalanceAfter = DEBT_TOKEN.balanceOf(address(this));
+       uint256 remainingCollateral = userCollAmount - params.swapAmount;
+       if (remainingCollateral > 0) {
+           _withdrawCollateral(remainingCollateral);
+           COLL_TOKEN.safeTransfer(msg.sender, remainingCollateral);
+       }
+
        uint256 withdrawableEquity = debtTokenBalanceAfter -
            debtTokenBalanceBefore;
```

© 2026 OffBytes Lab. All rights reserved.

5. About Us

About OffBytes Lab

OffBytes Lab is an independent blockchain security firm focused on smart-contract audits.

Our researchers come from competitive audit platforms such as Code4rena and Sherlock and have a track record of validated High- and Medium-severity findings across DeFi, bridge, staking and NFT protocols.

Every engagement combines rigorous manual review with executable proof-of-concept tests, so that each reported issue is demonstrated rather than theorised and each fix is re-verified against the same tests.

Audit Methodology

Our audit methodology is specialised to provide thorough security assessments of EVM smart contracts. We focus on rigorous manual analysis, detailed threat modelling, executable proof-of-concept validation, and careful review of implemented fixes.

1. Program Context and Architecture Analysis

We begin by studying the protocol's documentation and intended functionality, mapping contract interactions, transaction flows and state management. Special attention is given to how the protocol interfaces with common EVM standards such as ERC-20, ERC-721 and ERC-4626, with proxy and governance modules, and with external protocols, verifying that their inputs and return values are handled correctly.

2. Threat Modelling

We define attacker capabilities for the EVM execution environment and enumerate the vulnerability classes relevant to the codebase, including but not limited to:

- Improper access control, role checks or `msg.sender` verification
- Unauthorised state-variable modification
- Misuse of low-level calls, `delegatecall`, or proxy / upgradeability patterns (storage collisions, initialisation)
- Incorrect handling of external calls, including reentrancy vectors
- Denial-of-service vectors related to gas limitations or reverting dependencies

3. In-depth Manual Security Review

Our auditors perform a comprehensive line-by-line inspection of the source code, focusing on:

- Missing or insufficient access control and role checks
- Inadequate `msg.sender` and transaction-origin checks
- Incorrect handling of external calls and callback privileges, including reentrancy
- Arithmetic overflow / underflow and rounding errors
- Unsafe use of ABI encoding / decoding or low-level assembly
- Improper ERC-20 / token handling (approval logic, flash-loan vectors, token locking)
- Logic flaws in financial operations or state transitions

© 2026 OffBytes Lab. All rights reserved.

- Edge-case handling in function input validation
- Denial-of-service vectors related to gas usage and transaction execution

Each discovered vulnerability is documented with a description, precise severity rating and secure-implementation recommendation. Where exploitability is not obvious we include an executable proof-of-concept.

4. Detailed Fix Review and Validation

After the team's remediation we review each fix individually, confirming that corrections eliminate the security risk, do not introduce regressions or new vulnerabilities, and align with EVM best practices. Original proof-of-concept tests are re-run against the fixed code.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with OffBytes Lab. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorised by OffBytes Lab.

Legal Disclaimer

The review and this report are provided by OffBytes Lab on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, OffBytes Lab disclaims all warranties, expressed or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. OffBytes Lab does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgement and exercise caution where appropriate.

© 2026 OffBytes Lab. All rights reserved.